

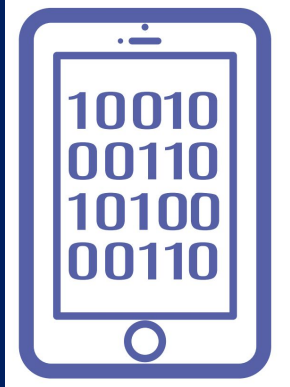
MOBİL UYGULAMA



İSTANBUL DENEYAP TEKNOLOJİ ATÖLYESİ

Ders Saatleri : 09.00 - 13.00 - Lise Sabah Grubu (Sınıfta) - (9 Temmuz- 16 Temmuz Hafta İçi)

Ders Saatleri : 13.30 - 17.30 - Orta Öğle Grubu (Sınıfta) - (9 Temmuz- 16 Temmuz Hafta İçi)



MOBİL UYGULAMA DERSİ

1.Ders:Mobil Uygulamalar-Flutter-Dart



T3 DENEYAP EĞİTMENİ (Hazırlayan ve Sunan): Muhammed Emin Korkunç

(Bilgiyar Mühendisi)

Bu sunumun kopyalanması, paylaşılması ve yazılı izinsiz kullanımı KESİNLİKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunc-100ba2215 | github.com/muhammedkorkunc

MOBİL UYGULAMA

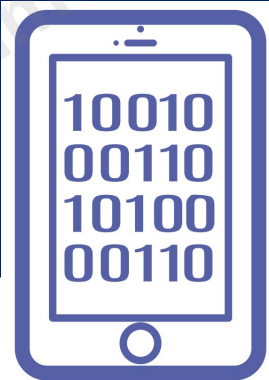


Sabah Grubu

- 09.00 - 09.50: 1. Ders
- 09.50 - 10.00: 1. Mola
- 10.00 - 10.50: 2. Ders
- 10.50 - 11.00: 2. Mola
- 11.00 - 11.50: 3. Ders
- 11.50 - 12.00: 3. Mola
- 12.00 - 13.00: 4. Ders

Öğle Grubu

- 13.30 - 14.20: 1. Ders
- 14.20 - 14.30: 1. Mola
- 14.30 - 15.20: 2. Ders
- 15.20 - 15.30: 2. Mola
- 15.30 - 16.20: 3. Ders
- 16.20 - 16.30: 3. Mola
- 16.30 - 17.30: 4. Ders



Ders Öncesi Gereksinimler 1. Bu derste anlatılacak uygulamalar için öğrencilerin bilgisayarlarında Flutter uygulama geliştirme ortamının kurulu olması gerekmektedir. Bu amaçla Windows ve MacOS işletim sistemlerinde yükleme işlemlerinin anlatıldığı videolar hazırlanmıştır. Öğrencilerin ders öncesinde bu videoları izleyerek kurulumları tamamlamış olmaları gerekmektedir. Windows:

https://youtu.be/--k9iz_XR7A

3. Bu derste projelerin yönetimi ve kod yazma işlemleri için VSCode düzenleyicisinin kullanılması önerilmektedir. Ücretsiz dağıtılan bu düzenleyici <https://code.visualstudio.com/download> adresinden edinilebilir.

Mobil Uygulama Geliştirme Eğitimi →

"Merhaba geleceğin teknoloji yıldızları! Bugün harika bir serüvene başlıyoruz. Hayatımızın her anında elimizde olan o akıllı telefonları ve giyilebilir teknolojileri asıl akıllı yapan şey nedir biliyor musunuz? Tabii ki içindeki uygulamalar! Önümüzdeki günlerde Flutter ekosistemini öğrenerek kendi uygulamalarımızı geliştireceğiz. Hazırsanız başlayalım!"

Sabah uyandığınızdan şu ana kadar telefon veya tablete dokunup en az bir uygulama açmayan var mı? Varsa hangi uygulamaydı bu?

2- Mobil Uygulama Nedir? 🤔

Mobil Uygulama Ekosistemi 🌐

Mobil uygulamalar; akıllı telefon, tablet ve akıllı saat gibi küçük, taşınabilir ve kablosuz cihazlar için özel olarak üretilmiş yazılımlardır. Masaüstü bilgisayarlar çok güçlüdür ama onları cebimizde taşıyamayız, değil mi? İşte mobil cihazlar bu yüzden hayatımızın merkezinde!

📱 Güçlü ve Zayıf Yönler:

-  **Dezavantajı:** Bilgisayarlara göre daha düşük işlem gücü ve kısıtlı batarya (pil) kapasitesi vardır. Geliştirici olarak pili sömürmeyen kodlar yazmalıyız!
-  **Avantajı:** Sürekli yanımızda olmaları, kablosuz bağlantıları, parmak izi okuyucuları, kameraları ve konum algılayıcıları (GPS) harika avantajlardır!

3- Devler Yarışıyor: Kim Bu Pazarın Lideri?



Mobil Dünyanın İki Büyük Kralı

Dünyada mobil işletim sistemi denince akla iki büyük dev geliyor. Pazarın neredeyse tamamı bu iki isme ait!

 **Android (Google):** Yaklaşık %72'lik pazar payı ile dünya lideri! Uygulama mağazası: Play Store.

 **iOS (Apple):** Yaklaşık %26'lık pazar payı ile ikinci sırada! Uygulama mağazası: App Store.

Geriye kalan %2 ise diğer küçük girişimler arasında paylaşıyor."

Sizce neden başka bir marka (mesela Microsoft veya Samsung) bu iki devin yanına üçüncü bir büyük güç olarak yerleşemedi? Fikri olan var mı?"



4- Rakamlarla Mobil Uygulama Çılgınlığı



Sayılarla Mobil Pazar

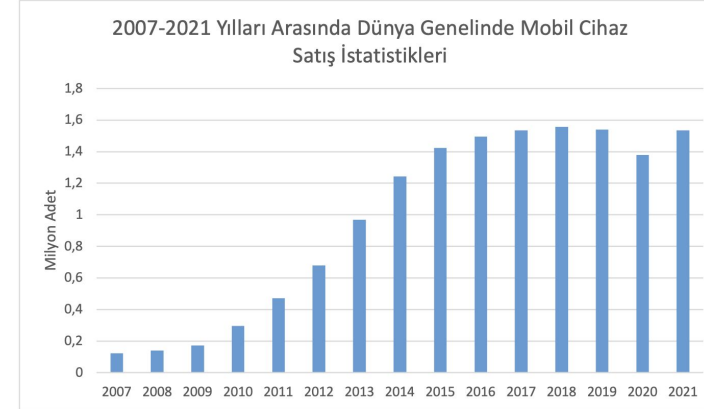


Bu sektörde o kadar büyük paralar ve indirmeler dönüyor ki inanamayacaksınız!

2016 yılında dünyada 149 milyar uygulama indirilirken, bu sayı kısa sürede 352 milyar adet üzerine çıktı!

Sadece 2021'in ilk yarısında insanların uygulamalara harcadığı para 64.9 milyar dolar!

En ilginç istatistik ise şu: İnsanlar mobil cihazlarda geçirdikleri vaktin %89.2'sini sadece uygulamaların içinde geçiriyor! Yani tarayıcıda gezinmek yerine direkt uygulamaları açıyoruz."



Grafik 1. 2007-2021 yılları arasında dünya genelinde mobil cihaz satış istatistikleri (Kaynak: O'dea, 2021)

Mobil Dünyada Yeni Trendler ✨

Dünya çok hızlı değişiyor. Bugün başarılı bir uygulama yapmak istiyorsak şu trendleri bilmeliyiz:

Alışveriş artık tamamen mobile kaydı. Tek tuşla sipariş ve sesli alışveriş çok popüler.

 **Platform Bağımsız (Hibrit) Uygulamalar:** Hem Android hem de iOS için ayrı ayrı kod yazmak yerine, tek bir kod yazıp her yerde çalıştırıyoruz. İşte biz de bu yüzden Flutter öğreneceğiz!

 **Beacon (İşaretleyici) Teknolojisi:** Mağazaların önüne geldiğinizde telefonunuza anında 'Şu an indirimdeyiz!' bildirimi atan o küçük bluetooth cihazlarını hiç duydunuz mu?"

"Bir mağazanın yanından geçerken telefonunuza o mağazayla ilgili bir kampanya mesajı gelse bu hoşunuza mı gider, yoksa sizi rahatsız mı eder? Neden?"

Geleceğin Mobil Deneyimi ✨

Geleceğin teknolojileri şimdiden kapımızda:

 **Katlanabilir Ekranlar:** Telefonu açınca ekran iki katına çıkıyor! Uygulamalarımızı tasarlarken bu ekran dönüşümlerine uyumlu yapmalıyız.

 **Mobil Cüzdanlar:** NFC ve sesli ödeme ile artık fiziksel cüzdanlar tarih oluyor.

 **Giyilebilir Cihazlar:** Akıllı saatler, akıllı lensler ve vücut algılayıcıları ile sağlık verilerimiz anlık takip ediliyor.

 **Artırılmış (AR) ve Sanal Gerçeklik (VR):** Kameraları kullanarak odanın boyutunu ölçen ya da odamıza sanal bir mobilya yerleştiren uygulamalar artık çok yaygın!

1-"Eğer sadece giyilebilir cihazlar (akıllı saatler veya akıllı gözlükler) için bir uygulama geliştirecek olsaydınız, bu ne uygulaması olurdu? Hayatımızı nasıl kolaylaştırırdı?"

2-"Katlanabilir ekranlar için oyun tasarlasanız, ekran açıldığında oyunda nasıl bir sürpriz veya değişiklik olmasını isterdiniz?"

3-"Gelecekte mobil cüzdanların tamamen hayatımıza girmesiyle paranın şekli sizce nasıl değişecek?"

8- Geleceğin Ticari Fikirleri & Beyin Fırtınası!



Hangi Eğilimleri Takip Etmeliyiz? 💰

Geleceğin teknoloji liderleri! Az önce harika trendler gördük. Peki sizce bu trendlerden hangisinde en çok ticari potansiyel (yani çok satma ve para kazanma şansı) var?

***Giyilebilir Teknolojiler:* "Çocukların veya evcil hayvanların kaybolmasını önlemek için akıllı saatlere nasıl bir özellik ekleyebiliriz?"**

***Artırılmış Gerçeklik (AR):* "Alışveriş yaparken ya da odamızı düzenlerken AR bize nasıl yardımcı olabilir?"**

Mobil Uygulama Yaklaşımları

"Bir mobil uygulama geliřtirmek istediğimize önümüzde 3 farklı yol bulunur. Hepsinin kendine göre süper güçleri ve zayıf yönleri var.

Bunlar: Yerel (Native), Web ve Hibrit yaklaşımlar. Gelin bunları yakından tanıyalım!"

Ev Sahibi: Yerel (Native) Yaklaşım

"Uygulamayı, sadece o işletim sisteminin kendi resmi dilleri ve araçlarıyla yazmaktır.

 iOS için: Mac bilgisayarda, XCode programıyla Swift veya Objective-C dillerinde yazılır.

 Android için: Android Studio programıyla Java veya Kotlin dillerinde yazılır.

 Avantajı: Çok hızlı çalışırlar ve cihazın tüm yeni özelliklerini (yüz tanıma, parmak izi vb.) anında kullanabilirler.

 Dezavantajı: Birinde yazdığın kod diğerinde çalışmaz! İki ayrı pazar için iki ayrı uygulama yapmak gerekir. Bu da çok pahalı ve zaman alıcıdır "

11- 2. Web Uygulamaları

Mobil Görünümlü Web Siteleri

Özünde HTML5, CSS ve JavaScript teknolojileriyle yazılmış birer web sitesidir ama dışarıdan bakınca mobil uygulama gibi görünürler.

 Tarayıcı üzerinden telefonun ana ekranına bir ikon olarak eklenirler.

 **Avantajı:** Geliştirmesi çok hızlıdır. İçerik sunucuda durduğu için tek bir güncelleme ile tüm kullanıcıların uygulaması aynı anda güncellenir! Telefonda neredeyse hiç yer kaplamazlar.

 **Dezavantajı:** Cihazın derin güvenliği nedeniyle parmak izi algılayıcısı gibi donanımlara erişemeyebilirler ve çalışmak için çoğunlukla internete bağımlıdırlar."

Tek Ok ile İki Kuş: Hibrit Yaklaşım

"Piyasada binlerce çeşit Android ve iOS cihaz var. Hepsine ayrı ayrı yerel uygulama yapmak çok pahalı! İşte imdadımıza Hibrit (Platform Bağımsız) yaklaşım yetişiyor.

 Tek bir kod yazıyoruz, kullandığımız akıllı araçlar sayesinde bu kod hem Android hem de iOS paketine (APK / APP) dönüşüyor!

 İşte bizim bu derste öğreneceğimiz Flutter da tam olarak popüler bir Hibrit platformudur! (Diğerleri: React Native, Ionic)."

13- ETKİNLİK: Yaklaşımları Karşılaştıralım!



Büyük Karşılaştırma Tablosu 🔍

"Şimdi öğrendiklerimizi bu tabloda birleştirelim. Hangisi daha hızlı, hangisi daha ucuz beraber karar verelim!"

Ölçüt	📱 Yerel (Native) Uygulama	🌐 Web Uygulaması	🔄 Hibrit Uygulama	
Geliştirme Zamanı	⌚ Uzun (İki ayrı takım gerekir)	⚡ Çok Hızlı	⌚ Kısa (Tek kod tabanı)	
Maliyet	💰 Yüksek / Pahalı	🌐 Düşük / Ucuz	💰 Ekonomik / Orta	
Performans (Hız)	🚀 En Yüksek / Çok Hızlı	🐢 Düşük / Yavaş	⚡ Yüksek (Flutter ile çok hızlı)	
Donanıma Erişim	📡 Tam Erişim (Kamera, Sensör)	🔒 Kısıtlı Erişim	🔧 Geniş Erişim (Kütüphanelerle)	
İnternet Bağımlılığı	📶 Çevrimdışı çalışabilir	🌐 Genellikle internet şart	📶 Çevrimdışı çalışabilir	

14- TASARIM ZAMANI: Hayalindeki Uygulamayı Çiz!



Arayüz (Mockup) Tasarımı ve Proto.io

Kayıt formu adresi: <https://proto.io/en/signup/>

"Kod yazmaya başlamadan önce, uygulamamızın nasıl görüneceğini tasarlamalıyız. Kodları çalışmayan ama görüntüsü ve butonları yerinde olan bu tasarımlara Prototip (Mockup) diyoruz.

 1. Adım (Kağıt-Kalem): Sayfaları ve içindeki eylemleri maddeler halinde kağıda çiziyoruz.

 2. Adım (Dijital Tasarım): Kağıttaki çizimlerimizi proto.io sitesine aktarıyoruz. Giriş sayfası, şifre hatırlatma gibi en az 3 sayfa tasarlayıp 'Preview' butonuna basarak sanki gerçek bir uygulamamış gibi test edeceğiz!"

15- Sahneye Flutter ve Dart Çıkıyor! ✨

Flutter Nedir ve Nasıl Çalışır? 🛠️

"Gelelim kahramanımıza! Flutter, Google tarafından geliştirilen ve tek bir kodla harika yerel uygulamalar yapmamızı sağlayan bir araçtır.

⚡ **Hız Sırrı:** Diğer bazı hibrit araçlar gibi araya çevirici katmanlar koymaz. Flutter, yazdığımız kodları doğrudan telefonun işlemcisinin anlayacağı makine koduna çevirir. Bu yüzden süper hızlıdır!

🧠 **Geliştirme Kiti (SDK):** Kodlarımızı makine diline çeviren fabrikadır.

📖 **Geriye Kütüphaneler (Widgets):** Ekrandaki butonları, resimleri ve sayfaları oluşturan hazır tasarım kutularımızdır."

Flutter'ın Dili: Dart

Flutter uygulamalarını yazmak için Google'ın geliştirdiği Dart dilini kullanacağız. Dart, nesneye yönelik ve sıkı veri kontrollü bir dildir. Yani hata yaparsak bizi daha kodu çalıştırırken uyarır!

Flutter ile Neler Üretebiliriz?

-  Android Uygulamaları
-  iOS ve MacOS Uygulamaları
-  Linux Uygulamaları (Masaüstü)
-  WebApp (İnternet Tarayıcılarında Çalışan Uygulamalar)

Tartışma 🎤

Şimdi bilgisayarlarımızın başına geçip proto.io tasarımlarımıza başlamadan önce şu soruyu düşünelim:

"Eğer gelecekte çok büyük bir mobil uygulama şirketi kursaydınız; uygulamanızı Native (Yerel) olarak mı başlatırdınız, yoksa Flutter gibi Hibrit bir yöntemle mi? Neden?"

18- Sihirli Komut: İlk Uygulamamızı Oluşturuyoruz! ✨

İlk Flutter Uygulamanız: app1 🚀

- "Evet arkadaşlar, şimdi teori bitti, uygulama başlıyor! Kendi klasörümüzü oluşturduktan sonra komut satırını (Terminal) açıyoruz ve Flutter fabrikasına ilk emrimizi veriyoruz. Sırasıyla gitmek için ilk uygulamamızın adını app1 koyduk. Hazırsanız klavyeden şu sihirli kodu yazıp Enter'a basıyoruz:"

flutter create app2

🔍 Arka Planda Ne Oluyor?:

- Bu komutu yazdığınızda Flutter bizim için otomatik olarak koca bir klasör dolusu dosya üretiyor! Gelin şimdi bu projenin içine göz atalım.

İlk Uygulamayı WebApp Olarak Koşturmak ⚡

Oluşturduğumuz uygulamayı hemen görmek istiyoruz değil mi? Bunun için bilgisayarımızda Google Chrome tarayıcısının yüklü olması gerekiyor. Şimdi terminalden sırasıyla şu iki komutu yazarak uygulamamızı ayağa kaldıracacağız. Önce `cd app1` diyerek uygulamamızın içine giriyoruz, sonra da `flutter run` diyerek çalıştırıyoruz!"

Çalıştırma Komutları:

1. Proje klasörünün içine gir: `cd app2`
2. Uygulamayı başlat: `flutter run`

"Ekranda sağ alttaki mavi renkli artı (+) düğmesine her bastığımızda ortadaki sayıya ne oluyor? Hadi herkes ilk tıklamasını yapsın ve sayının kaç olduğunu söylesin!"

Kodlarımızı Nereye Yazacağız?

"Flutter projelerimizin içindeki dosyaları rahatça düzenlemek, renklendirmek ve yeni kodlar eklemek için VSCode düzenleyicisini kullanacağız.

 Tek yapmanız gereken, terminalde oluşturduğumuz app1 klasörünü tutup VSCode ekranının üzerine sürükleyip bırakmak!

 Sol taraftaki Explorer (Dosya Gezgini) butonuna tıkladığımızda projenin tüm organları önümüze serilecek. Bizim için en en önemli dosya hangisi biliyor musunuz? Tabii ki lib klasörünün altındaki main.dart dosyası! Tüm iş bu dosyada dönüyor."



Önemli Klasörleri Tanıyalım

"Yazılım geliştirme kiti (Y GK) bizim için bir sürü dosya üretti demiştik. Kafamız karışmasın, hepsinin çok eğlenceli görevleri var! Gelin birer benzetmeyle bakalım:"

Ana Klasörler ve Görevleri:

-  **android:** Burası gizli bir Android projesidir. Sonunda Android telefonu olanlara dağıtacağımız APK dosyası buradan çıkacak! Bizim pek dokunmamıza gerek yok, arka planda çalışır.
-  **ios:** Bu da işin Apple tarafı! İçinde bütün bir iOS projesi saklanır. (Windows bilgisayar kullananlar bu klasörü göremeyebilir çünkü Apple sadece kendi cihazlarında izin verir).
-  **build:** Kodlarımızın derlenip toplandığı yerdir. Flutter burayı kendisi yönetir, sakın buradaki dosyalara ellemeyelim!



En Çok Kullanacağımız Dosyalar ★

"Gelelim bizim asıl oyun alanlarımıza ve bilmemiz gereken diğer önemli dosyalara:"

❤️ lib (Library): Bizim en çok vakit geçireceğimiz kalbimiz! Yazacağımız tüm Dart kodları bu klasörün içine eklenecek.

🔧 pubspec.yaml: Uygulamamızın Ayar Belgesidir! Uygulamaya dışarıdan internetten müzik, yeni yazı tipleri (fontlar), ikonlar veya fotoğraflar eklemek istediğimizde bu dosyaya isimlerini tek tek kaydetmek zorundayız.

📖 README.md: Bu uygulama nedir, nasıl çalışır diye merak eden diğer yazılımcı arkadaşlarımız için bırakacağımız küçük bir günlük veya tanıtım yazısıdır.

Flutter'ın Kendi Takip Dosyaları

"Yazılımcı olarak bizim hiç dokunmayacağımız, Flutter'ın kendi kendine oluşturup yönettiği dosyalar da var. Bunları sadece isim olarak bilseniz yeterli:"

Dokunulmaz Dosyalar Listesi:

-  **test:** İleride uzmanlaştığımızda uygulamamızın hatalarını otomatik test eden robot kodların durduğu yerdir.
-  **.gitignore:** Kodlarımızın günlük yedeğini alan Git sisteminin neleri görmezden geleceğini yazar.
-  **.metadata & .packages:** Flutter'ın arka plandaki üst verilerini ve kütüphane yollarını tutar.
-  **pubspec.lock & app1.iml:** Ayarlarımızın kilitlendiği ve projelerin gereksinimlerinin takip edildiği dosyalardır.



Hadi Kodları İnceleyelim!

"Şimdi herkes bilgisayarında app1 klasörünü açıp VSCode üzerinden `lib/main.dart` dosyasını incelesin. Kodlar ilk başta çok karmaşık görünebilir ama dikkatli bakarsanız İngilizce bazı kelimeler göreceksiniz."

"Şu an önünüzde açık olan `main.dart` dosyasının içinde gezinerek bana ekrandaki 'Mavi butonun sayacını artıran' veya 'Uygulamanın başlığını tutan' bir kelime bulup gösterebilecek ilk dedektif kim olacak?"



25- Kodumuzun Röntgenini Çekiyoruz!



main.dart Dosyasının Anatomisi

"Geleceğin yazılımcıları, az önce karmaşık gelen o **main.dart** dosyasını tamamen temizledik ve ekrana sadece 'Merhaba!' yazan en sade haline getirdik! Şimdi bu 11 satırlık sihirli kodun röntgenini çekeceğiz. Her satırın ne işe yaradığını öğrendiğinizde, kod yazmak size legolarla oynamak gibi gelecek!"

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(MyApp());  
}
```

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        body: Center(  
          child: Text("Merhaba!"),  
        ),  
      ),  
    );  
  }  
}
```

27- import Satırı Ne İşe Yarar?

Dart:

```
import 'package:flutter/material.dart';
```

- "İlk satırımızda `import` yani 'içeri aktar' komutunu görüyoruz. Bunu lego kutusunu masaya dökmek gibi düşünebilirsiniz. Flutter'ın bizim için hazırladığı hazır butonlar, renkler ve yazı şekilleri bu `material.dart` isimli kütüphanenin içinde saklanıyor.
-  Eğer bu satırı silersek ne olur? Aşağıda yazacağımız neredeyse her kelimenin altı kırmızı çizgiyle çizilir, yani kodumuz hata verir! Çünkü bilgisayar o kelimelerin ne anlama geldiğini bulamaz."

28- Satır 3-5: Motoru Çalıştır: main() Fonksiyonu

Uygulamanın Başlangıç Noktası

Dart :

```
void main() {
```

```
  runApp(MyApp());
```

- "Bir uygulamanın çalışmaya başladığı ilk ana kapı main() fonksiyonudur. Dart dili, ilk olarak bu ismi arar ve motoru buradan çalıştırır.
-  void ne demek? Dart çok disiplinli bir dildir, fonksiyonların geriye bir şey döndürüp döndürmeyeceğini bilmek ister. void, 'bu fonksiyon sadece işini yapar, geriye ekstra bir sayı veya veri hesaplayıp döndürmez' demektir.
-  runApp(MyApp()): İşte bu komut, Flutter'a 'Hadi bakalım, ekrana MyApp isimli tasarımımızı çizmeye başla!' emrini verir."

Bileşen Ağacı Nedir?

- "Flutter dünyasında ekranda gördüğünüz HER ŞEY birer Widget'tır (yani bileşendir).
-  Uygulamanın kendisi, üstteki başlık çubuğu, yazılar, resimler, butonlar... Hepsi birer yapım parçasıdır.
- Bu parçalar alt alta, yan yana birleşerek bir Bileşen Ağacı oluşturur ve uygulamamızın ekranını meydana getirir."

```
class MyApp extends StatelessWidget {
```

"Burada `class` anahtar kelimesiyle kendi uygulamamızın şablonunu oluşturmaya başlıyoruz.

 `extends StatelessWidget` (Kalıtım): İşte en havalı kısım! Biz sıfırdan ekrandaki pikselleri boyamak için binlerce satır kod yazmıyoruz. Flutter'ın hazır `StatelessWidget` sınıfından miras (`extends`) alıyoruz. Onun tüm süper güçleri bir anda bizim `MyApp` sınıfımıza geçiyor!

 `StatelessWidget` Nedir? Ekranda sabit, kendi kendine değişmeyen içerikleri (örneğin düz bir yazıyı) göstermek için kullanılan bileşen türüdür."

31- Satır 8-10: Ekranı Boyama Zamanı: build Metodu

Ekranı Ne Çizeceğiz? 

```
8 Widget build(BuildContext context) {
```

```
9   return MaterialApp(home: Text("Merhaba!"));
```

```
10 }
```

"Miras aldığımız sınıf bize der ki: 'Sana tüm güçlerimi verdim ama ekrana ne çizeceğimi bana build metodunun içinde söylemek zorundasın!'"

 return: 'İşte ekrana bunu döndür/çiz' anlamına gelir.

 **MaterialApp & Text:** Önce genel bir mobil uygulama çerçevesi (MaterialApp) oluşturuyoruz. Onun ana sayfasına da (home:) ekranda kocaman "Merhaba!" yazması için bir Text bileşeni yerleştiriyoruz."



Pascal ve Camel Sözdizimi Nedir? 🐪

"Yazılımcılar kod yazarken kelimelerin arasına asla boşluk bırakamazlar. Kodun okunabilmesi için dünyaca kabul edilmiş iki harika yöntem kullanırlar:

🐪 **Camel (Deve) Sentaksı (Fonksiyonlar için):** İlk kelimenin ilk harfi küçük, sonraki gelen kelimelerin baş harfi büyüktür. Tıpkı devenin hörgüçleri gibi!

- **Örnek:** `birinciFlutterFonksiyonum`

📐 **Pascal Sentaksı (Sınıf isimleri için):** İstisnasız her kelimenin baş harfi büyüktür!

- **Örnek:** `MerhabaUygulamasi` veya bizim kodumuzdaki gibi `MyApp`"



Bugün bir Flutter kodunun kalbini ve anatomisini tamamen çözdünüz. Kapanıştan önce yazılımcıların dilindeki o iki meşhur kelimenin farkına bakalım:

Sınıfın içine yazılan fonksiyonlara Metot denir, tek başına duranlara ise Fonksiyon denir.

Muhammed Emin Korkunç

Uygula: `main.dart` Dosyasını Düzenleyelim 

Evet dedektifler ve yazılımcılar, şimdi sıra sizde! İlk oluşturduğumuz o hazır sayaç uygulamasının kodlarını açıyoruz ve ekranı tamamen kendimize göre özelleştiriyoruz. Kodun içinde aşağı doğru indikçe bu kelimeleri bulup, yerine yeni Türkçe karşılıklarını yazacağız. Bakalım kimler hatasız tamamlayacak?

-  a. Tarayıcı sekmesindeki başlığı **title**: `'Tıklama Sayacı'` yapıyoruz.
-  b. Mavi barın içindeki yazıyı **title**: `Text('Tıklama Sayacı Uygulaması')` yapıyoruz.
-  c. Sayacın başlangıç değerini **int** `_counter = 0`; olan yerden 1'e eşitliyoruz (`= 1`;;).
-  d. Ortadaki mesajı `'Düymeye şu kadar kez bastınız:'` olarak değiştiriyoruz.
-  e. En alttaki butonun üzerine gelince çıkan yazıyı **tooltip**: `'Arttır'` yapıyoruz.

35- İşte Düzenlenmiş Yeni Kodumuz!



Değiştirmeniz Gereken Satırlar (`lib/main.dart`) ⚙️

`lib/main.dart` dosyanızın içindeki eski kodları tamamen silip, yerine bu düzenlenmiş kodları yapıştırabilirsiniz. Değiştirdiğimiz yerleri sizin için yanlarına yeşil açıklamalarla ekledim, dikkatlice inceleyin!

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(const MyApp());  
}
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});
```

```
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Tıklama Sayacı', // (a) Sekme başlığı düzenlendi  
      theme: ThemeData(  
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),  
        useMaterial3: true,  
      ),  
      home: const MyHomePage(),  
    );  
  }  
}
```

```
class MyHomePage extends StatefulWidget {  
  const MyHomePage({super.key});
```

```
  @override  
  State<MyHomePage> createState() => _MyHomePageState();  
}
```

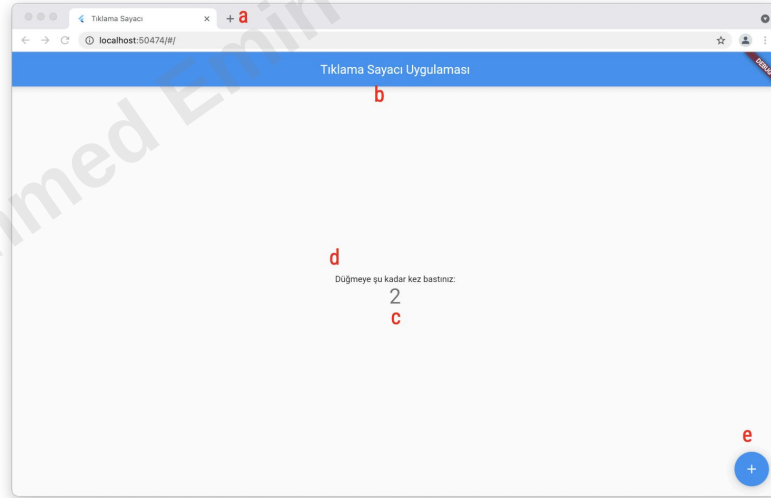
```
class _MyHomePageState extends State<MyHomePage> {  
  int _counter = 1; // (c) Sayaç 1'den başlatıldı!
```

```
  void _incrementCounter() {  
    setState(() {  
      _counter++;  
    });  
  }
```

```
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        backgroundColor: Colors.blue,  
        title: const Text('Tıklama Sayacı Uygulaması', style: TextStyle(color: Colors.white)), // (b) Başlık çubuğu düzenlendi  
      ),  
      body: Center(  
        child: Column(  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: <Widget>[  
            const Text(  
              'Düymeye şu kadar kez bastınız', // (d) Ortadaki mesaj düzenlendi  
            ),  
            Text(  
              '$_counter',  
              style: Theme.of(context).textTheme.headlineMedium,  
            ),  
          ],  
        ),  
      ),  
      floatingActionButton: FloatingActionButton(  
        onPressed: _incrementCounter,  
        tooltip: 'Arttır', // (e) Yardım metni düzenlendi  
        child: const Icon(Icons.add),  
      ),  
    );  
  }  
}
```

Bu sunumun kopyalanması, paylaşılması ve yazılı izinsiz kullanımı KESİNLİKLE YASAKTIR.

Kodları kaydedip terminale tekrar flutter run yazıyoruz veya tarayıcımızı yeniliyoruz! Eğer her şey doğruysa ekrandaki gibi bir görüntüye ulaşmış olmanız gerekiyor.



Ekran Görüntüsü 6. Değerlendirme ödevi arayüzü

<https://create.kahoot.it/share/mobil-uygulama-1-ders/017bb52c-8a52-419d-a1bf-14f2ec8122cc>

